LIBXSTREAM

LIBXSTREAM is an OpenCL-based accelerator backend library providing streams, events, and device memory management for GPU offloading. It implements the DBCSR ACC interface, making it a drop-in OpenCL backend for CP2K/DBCSR.

Targets OpenCL devices across vendors (Intel, AMD, NVIDIA). Depends on LIBXS for utility infrastructure.

Build

LIBXS must be a sibling directory (controlled by `LIBXSROOT`). An OpenCL SDK must be available.

Library — build both LIBXS and LIBXSTREAM as separate libraries:

```
git clone https://github.com/hfp/libxs.git
git clone https://github.com/hfp/libxstream.git

cd libxs && make -j $(nproc)
cd ../libxstream && make -j $(nproc)
```

This produces `lib/libxstream.a` and `lib/libxstream.so`.

Header-only (explicit) — include `libxstream_source.h` (no separate library needed for either LIBXSTREAM or LIBXS). Safe to include from multiple translation units:

```
#include <libxstream/libxstream_source.h>
```

Header-only (implicit) — compile with `-DLIBXSTREAM_SOURCE`. Any LIBXSTREAM or LIBXS public header then automatically includes the implementation. No special include order is required. `-DLIBXSTREAM_SOURCE` implies `-DLIBXS_SOURCE`; LIBXS can also be made header-only independently with `-DLIBXS_SOURCE`.

The library is compiled for SSE4.2 by default but dynamically dispatches to the best ISA available at runtime (up to AVX-512). Use `SSE=0` to compile natively for the build host.

Variable	Default	Description
GNU	1	GNU GCC-compatible compiler (0: Intel)
DBG	0	Debug build
SYM	0	Include debug symbols (-g)
SSE	1	x86 baseline: 0=native, 1=SSE4.2 (portable)

pkg-config support: `lib/libxstream.pc`.

Installation

Install into a chosen prefix (LIBXS must be built first):

```
make -j $(nproc) install PREFIX=$HOME/libxstream
```

This installs headers and the static and shared libraries under `PREFIX`. The header-only source tree is optional; add `HEADER_ONLY=1` to install it.

Out-of-tree builds are also supported:

```
mkdir /tmp/libxstream-build && cd /tmp/libxstream-build
make -j $(nproc) -f /path/to/libxstream/Makefile
```

API

The public C API is declared in `libxstream/libxstream.h`. All implementation details are sealed behind opaque types.

Initialization

```
int libxstream_init(void);
int libxstream_init_config(const libxstream_init_config_t* cfg);
void libxstream_init_config_default(libxstream_init_config_t* cfg);
int libxstream_finalize(void);
```

`libxstream_init()` initializes with environment/defaults. `libxstream_init_config()` allows explicit control over USM level, device selection, and verbosity before the runtime starts:

```
libxstream_init_config_t cfg;
libxstream_init_config_default(&cfg);  /* -1 = use env/default */
cfg.usm = 1;      /* 0:off, 1:Intel USM, 2:SVM coarse, 3:SVM caps */
cfg.device = 0;   /* device index (-1: env/first) */
cfg.verbosity = 2;
libxstream_init_config(&cfg);
```

Passing NULL to `libxstream_init_config` is equivalent to calling `libxstream_init`.

Devices

```
int libxstream_device_count(int* ndevices);
int libxstream_device_set_active(int device_id);
int libxstream_device_sync(void);
```

Streams

```
int libxstream_stream_create(libxstream_stream_t** stream_p,
                             const char* name, int priority);
int libxstream_stream_destroy(libxstream_stream_t* stream);
int libxstream_stream_sync(libxstream_stream_t* stream);
int libxstream_stream_wait_event(libxstream_stream_t* stream,
                                 libxstream_event_t* event);
```

Events

```
int libxstream_event_create(libxstream_event_t** event_p);
int libxstream_event_destroy(libxstream_event_t* event);
int libxstream_event_record(libxstream_event_t* event,
                            libxstream_stream_t* stream);
int libxstream_event_query(libxstream_event_t* event,
                           libxstream_bool_t* has_occurred);
int libxstream_event_sync(libxstream_event_t* event);
```

Memory

Device and host memory allocation, transfers (H2D, D2H, D2D), and initialization:

```
int libxstream_mem_allocate(void** dev_mem, size_t nbytes);
int libxstream_mem_deallocate(void* dev_mem);
int libxstream_mem_host_allocate(void** host_mem, size_t nbytes,
                                 libxstream_stream_t* stream);
int libxstream_mem_host_deallocate(void* host_mem,
                                   libxstream_stream_t* stream);
int libxstream_mem_copy_h2d(const void* host_mem, void* dev_mem,
                            size_t nbytes, libxstream_stream_t* stream);
int libxstream_mem_copy_d2h(const void* dev_mem, void* host_mem,
                            size_t nbytes, libxstream_stream_t* stream);
int libxstream_mem_copy_d2d(const void* src, void* dst,
                            size_t nbytes, libxstream_stream_t* stream);
int libxstream_mem_zero(void* dev_mem, size_t offset, size_t nbytes,
                        libxstream_stream_t* stream);
```

The `_hint` variants take `LIBXSTREAM_MEM_POOLED` or `LIBXSTREAM_MEM_UNPOOLED` (`libxstream_mem_pool_t`), optionally combined with a `libxstream_opencl_mem_hint_t`. Unpooled memory suits a caller that pools on its own; releasing it synchronizes the device first. Deallocation takes the hint given at allocation:

```
int libxstream_mem_dev_allocate_hint(void** dev_mem, size_t nbytes, int hint);
int libxstream_mem_dev_deallocate_hint(void* dev_mem, int hint);
int libxstream_mem_host_allocate_hint(void** host_mem, size_t nbytes, int hint);
int libxstream_mem_host_deallocate_hint(void* host_mem, int hint);
```

DBCSR Compatibility

The header `libxstream/libxstream_dbcsr.h` provides the `c_dbcsr_acc_*` symbols expected by DBCSR, allowing LIBXSTREAM to serve as a drop-in accelerator backend.

CP2K Offload Interface

The header `libxstream/libxstream_cp2k.h` implements CP2K's offload runtime interface — the `offload*` functions for general GPU operations (memory, streams, events, synchronization).

Samples

Each sample has its own Makefile under `samples/`:

```
cd samples/smm && make
cd samples/ozaki && make
```

SMM — Small Matrix Multiplication

Implements the ACC LIBSMM interface for batched small matrix multiply and transpose on OpenCL devices. Includes an auto-tuning framework and pre-tuned parameter sets for A100, BMG, GH200, H100, Mi250, P100, PVC, and V100. See samples/smm/README.md.

Ozaki — High-Precision GEMM

Ozaki scheme for high-precision GEMM emulation, fully offloaded to OpenCL. Two schemes (mantissa slicing and CRT) with automatic detection of Intel XMX matrix engines. See samples/ozaki/README.md. The CPU-side GEMM wrapper is part of LIBXS Ozaki.

Stencil — BF16-DPAS Finite Difference

Seismic wave propagation (RTM/FWI) via dimension-split stencils computed as batched small GEMMs on BF16 DPAS/XMX tensor units. Achieves FP32 accuracy through Dekker splitting. Supports isotropic and TTI operators, multiple factorization methods (sparse, dense cascade, hybrid), and runtime kernel specialization via a thread-safe registry. Includes a scalar proxy for non-DPAS hardware. See samples/stencil/README.md.

License

BSD 3-Clause

LIBXSTREAM Domains

CP2K Offload Interface

`libxstream/libxstream_cp2k.h` implements CP2K's offload runtime interface — the hardware-abstraction layer that CP2K uses for GPU-accelerated operations beyond DBCSR (grid integration, PW operations, etc.). The original interface provides `static inline` wrappers for CUDA, HIP, and OpenCL; LIBXSTREAM replaces the OpenCL path with a dedicated translation unit (`src/libxstream_cp2k.c`) that routes directly through LIBXSTREAM's internal API.

Relationship to the DBCSR Interface

CP2K's code has two accelerator interfaces:

Interface	Header	Purpose
DBCSR ACC	`libxstream_dbcsr.h`	Sparse matrix operations (DBCSR library)
Offload Runtime	`libxstream_cp2k.h`	General offload (memory, streams, events, synchronization)

The DBCSR adapter (`src/libxstream_dbcsr.c`) uses opaque `void*` handles and translates to LIBXSTREAM's typed API. The offload runtime adapter does the same but uses CP2K's `offloadStream_t`/`offloadEvent_t` typedefs (also `void*`). Both share the underlying LIBXSTREAM implementation.

API

The header requires only `libxstream.h` (C99, no OpenCL headers) and provides opaque handle types, an error-checking macro, and functions covering five domains.

```
typedef void* offloadStream_t;
typedef void* offloadEvent_t;
typedef int   offloadError_t;

#define offloadSuccess EXIT_SUCCESS
```

```
const char* offloadGetErrorName(offloadError_t error);
offloadError_t offloadGetLastError(void);
```

`offloadGetErrorName` maps error codes to OpenCL error strings via `libxstream_opencl_strerror`. `offloadGetLastError` consumes and clears the last recorded error.

The `OFFLOAD_CHECK` macro aborts on failure after printing the error name and source location:

```
OFFLOAD_CHECK(offloadGetLastError());
```

The `offload*` functions return nothing; one that fails prints the OpenCL error name and code to stderr and aborts.

```
void offloadStreamCreate(offloadStream_t* stream);
void offloadStreamDestroy(offloadStream_t stream);
void offloadStreamSynchronize(offloadStream_t stream);
void offloadStreamWaitEvent(offloadStream_t stream, offloadEvent_t event);
```

`offloadStreamCreate` creates a stream with default priority (`LIBXSTREAM_STREAM_DEFAULT`).

```
void offloadEventCreate(offloadEvent_t* event);
void offloadEventDestroy(offloadEvent_t event);
void offloadEventRecord(offloadEvent_t event, offloadStream_t stream);
void offloadEventSynchronize(offloadEvent_t event);
bool offloadEventQuery(offloadEvent_t event);
```

```
void offloadMalloc(void** ptr, size_t size);
void offloadFree(void* ptr);
void offloadMallocHost(void** ptr, size_t size);
void offloadFreeHost(void* ptr);
```

These allocate from LIBXSTREAM's memory pools. CP2K's own pool (`offload_mempool.c`) allocates through the hinted functions with `LIBXSTREAM_MALLOC_CP2K` instead (default `LIBXSTREAM_MEM_UNPOOLED`), so that its pool is not stacked on LIBXSTREAM's.

```
void offloadMemcpyAsyncHtoD(void* ptr_dev, const void* ptr_hst, size_t size, offloadStream_t stream);
void offloadMemcpyAsyncDtoH(void* ptr_hst, const void* ptr_dev, size_t size, offloadStream_t stream);
void offloadMemcpyAsyncDtoD(void* dst, const void* src, size_t size, offloadStream_t stream);
void offloadMemcpyHtoD(void* ptr_dev, const void* ptr_hst, size_t size);
void offloadMemcpyDtoH(void* ptr_hst, const void* ptr_dev, size_t size);
void offloadMemsetAsync(void* ptr, int val, size_t size, offloadStream_t stream);
void offloadMemset(void* ptr, int val, size_t size);
```

The synchronous variants (`offloadMemcpyHtoD`, `offloadMemcpyDtoH`, `offloadMemset`) pass a NULL stream. `offloadMemsetAsync` supports arbitrary fill values via `libxstream_opencl_memset`.

```
void offloadDeviceSynchronize(void);
```

```
void offloadMemcpyToSymbol(const void* symbol, const void* src, size_t count);
void offloadEnsureMallocHeapSize(size_t required_size);
```

These are CUDA-specific operations (constant-memory writes and device-heap sizing) that have no direct OpenCL equivalent. They are currently stubs guarded by assertions. CP2K's OpenCL path disables the GPU grid subsystem (`__NO_OFFLOAD_GRID`) that would call them.

See Also

- LIBXSTREAM API (`libxstream/libxstream.h`) — the underlying OpenCL backend API
- DBCSR ACC Interface — the DBCSR adapter layer
- CP2K offload_runtime.h — upstream interface definition

DBCSR ACC Interface

`libxstream/libxstream_dbcsr.h` implements the DBCSR ACC interface — the accelerator backend contract defined by the DBCSR sparse-matrix library used in CP2K. By providing this interface on top of LIBXSTREAM's OpenCL runtime, the library acts as a **drop-in OpenCL backend** for DBCSR: no changes to DBCSR or CP2K source code are required.

Purpose

DBCSR expects every accelerator backend to expose a flat C API with the `c_dbcsr_acc_*` prefix covering five domains:

Domain	Functions
Initialization	`c_dbcsr_acc_init`, `c_dbcsr_acc_finalize`
Devices	`c_dbcsr_acc_get_ndevices`, `c_dbcsr_acc_set_active_device`, `c_dbcsr_acc_device_synchronize`
Streams	`c_dbcsr_acc_stream_create`, `c_dbcsr_acc_stream_destroy`, `c_dbcsr_acc_stream_sync`, `c_dbcsr_acc_stream_wait_eve`
Events	`c_dbcsr_acc_event_create`, `c_dbcsr_acc_event_destroy`, `c_dbcsr_acc_event_record`, `c_dbcsr_acc_event_query`, `c_d`
Memory	`c_dbcsr_acc_dev_mem_allocate`, `c_dbcsr_acc_dev_mem_deallocate`, `c_dbcsr_acc_dev_mem_set_ptr`, `c_dbcsr_acc_host`

Every function delegates to the corresponding `libxstream_*` routine (e.g., `c_dbcsr_acc_stream_create` calls `libxstream_stream_create`), translating between DBCSR's opaque `void*` handles and LIBXSTREAM's typed `libxstream_stream_t` / `libxstream_event_t` pointers.

Profiling

The header also declares `c_dbcsr_timeset` and `c_dbcsr_timestop`, which are DBCSR's Fortran-side timer callbacks. When profiling is enabled (`LIBXSTREAM_PROFILE_DBCSR`), every ACC function is bracketed by these calls so that individual backend operations appear in DBCSR's timing report.

Utility Macros

Macro	Description
`DBCSR_STRINGIFY(SYMBOL)`	Stringifies a preprocessor token
`DBCSR_CONCATENATE(A, B)`	Concatenates two preprocessor tokens
`DBCSR_MARK_USED(x)`	Silences unused-variable warnings

See Also

- LIBXSTREAM API (`libxstream/libxstream.h`) — the underlying OpenCL backend API
- DBCSR ACC specification — upstream interface definition

Visit LIBXS

OpenCL Backend

`libxstream/libxstream_opencl.h` is the internal OpenCL layer that powers every public `libxstream_*` function. It owns the OpenCL platform/device/context lifecycle, memory management, kernel compilation, and error handling. Sample code and other LIBXSTREAM extensions (e.g., LIBSMM, Ozaki) include this header to access the OpenCL runtime directly.

Compile-Time Configuration

The header is guarded by `__OPENCL` (set automatically when `__OFFLOAD_OPENCL` is defined). Key compile-time knobs:

Macro	Default	Description
`LIBXSTREAM_MAXALIGN`	2 MB	Maximum alignment for device allocations
`LIBXSTREAM_BUFFERSIZE`	8 KB	Internal scratch-buffer size
`LIBXSTREAM_MAXSTRLEN`	48	Maximum string length for names
`LIBXSTREAM_MAXNDEVS`	64	Maximum number of OpenCL devices
`LIBXSTREAM_MAXNITEMS`	1024	Per-thread maximum item count
`LIBXSTREAM_MAXNKERNELS`	32	Maximum number of distinct kernels that can be profiled
`LIBXSTREAM_PROFILE_TICKS`	10	Device-timer ticks a sample must span to be recorded
`LIBXSTREAM_USM`	SVM coarse-grain	Runtime Unified Shared Memory level (unset = OpenCL 2.0 SVM coarse-
`LIBXSTREAM_SUBBUFFER`	0	Sub-buffers for offset kernel-arguments (0 = off, 1 = on); unused where US

Levels 1 and 3 are opt-in and never reached by the default, even though both are faster in a microbenchmark. Level 1 (`cl_intel_unified_shared_memory`) is the only path with a genuinely asynchronous transfer — `clEnqueueMemcpyINTEL` measured 45.3 GB/s against 9.1 GB/s for a 128 MB H2D on a GPU Max 1550, where every SVM path instead ends in a host `memcpy` that runs single-threaded inside the enqueue and cannot overlap a kernel. It stays opt-in regardless, because the default must behave predictably across drivers rather than peak on one; request it explicitly where it is known good. A warning is emitted at verbosity 2+ if an explicit level 1 cannot load the extensions, since the only symptom is a slower run.

Level 3 is excluded from the default for a different reason. It adopts whatever `CL_DEVICE_SVM_CAPABILITIES` reports, and that varies by kernel driver: on i915 a GPU Max 1550 offers only coarse-grain buffers, whereas Xe also reports fine-grain, including fine-grain *system* allocations. That is a substantially broader contract than coarse-grain buffers and not something to acquire implicitly from a capability bit, so levels below 3 mask the capabilities down to `CL_DEVICE_SVM_COARSE_GRAIN_BUFFER`.

Nothing is lost by that: the grain is not a performance lever. Coarse-grain adds a `clEnqueueSVMMap`/`clEnqueueSVMUnmap` pair that fine-grain omits, but that is bookkeeping rather than data movement and the `memcpy` both paths end in dominates. Measured on a Xeon Platinum 8480+ (the device here reporting both grains), 64 MB H2D: 15.2 GB/s forced coarse against 15.1 GB/s with fine-grain.

On NVIDIA the capability is not queried by default, so the default level leaves USM off and no device memory pool is created. An explicitly requested level is honoured there, to exercise the path rather than to speed one up: results are correct and an H100 PCIe runs 6.7x slower that way. Leave it unset on that vendor.

Data Types

libxstream_opencl_config_t The central singleton (libxstream_opencl_config) populated by libxstream_init. It holds:

- **Device table** — ordered array of discovered cl_device_id entries.
- **Active device** (libxstream_opencl_device_t) — context, default stream, error slot, OpenCL standard level, workgroup limits, memory caps, vendor flags, and optional USM function pointers.
- **Resource pools** — lock objects, streams, events, memory-pointer registrations, and a host-memory pool (libxs_malloc_pool_t).
- **Runtime switches** — verbosity, async mode, debug/dump level, profiling, execution hints, and workaround level.
- **Histograms** — optional per-kernel duration histograms, and transfer-time histograms for H2D, D2H, D2D, and zero-fill operations.

libxstream_opencl_stream_t / libxstream_event_t Thin wrappers around cl_command_queue and cl_event respectively. Streams additionally carry a thread-ID and optional priority.

libxstream_opencl_info_memptr_t Associates a cl_mem buffer object with its host-side pointer, used to translate between SVM/USM pointers and buffer-based memory.

libxstream_opencl_atomic_fp_t Enumerates floating-point atomics support: none, 32-bit, or 64-bit.

Error Handling Macros

Macro	Description
CL_CHECK(RESULT, CALL)	Execute an OpenCL call; on failure record the error code and human-readable name
CL_ERROR_REPORT(NAME)	Print the last error to stderr (if verbosity is enabled)
CL_RETURN(RESULT, NAME)	Return from function, reporting the error if non-zero

Key Functions

Device and Context

Function	Description
libxstream_opencl_set_active_device	Internal device activation (lock-aware)
libxstream_opencl_create_context	Create an OpenCL context for a given device
libxstream_opencl_device_name	Return device name, platform name, and UID
libxstream_opencl_device_level	Query OpenCL version and device type
libxstream_opencl_device_vendor	Confirm a device's vendor string
libxstream_opencl_device_ext	Check for required OpenCL extensions
libxstream_opencl_device_uid	Capture or compute a unique device identifier
libxstream_opencl_device_match	Select the entry of a list of device names a device matches
libxstream_opencl_info_devmem	Query free/total/local device memory

Memory

Function	Description
libxstream_opencl_info_devptr	Look up a device-pointer registration (read-only)
libxstream_opencl_info_devptr_modify	Look up a device-pointer registration (writable)
libxstream_opencl_info_hostptr	Look up a host-pointer registration
libxstream_opencl_memset	Fill device memory with an arbitrary byte pattern
libxstream_opencl_use_cmem_size	Whether OpenCL constant-memory hints apply
libxstream_opencl_set_kernel_ptr	Set a pointer kernel argument (USM-aware)

CUDA Host Registration Host memory from `libxstream_mem_host_allocate` is pinned by OpenCL, which a CUDA context cannot see: a transfer issued by CUDA from such a buffer is pageable and several times slower. Where the application links the CUDA runtime, `cudaHostRegister`/`cudaHostUnregister` are resolved at startup (global scope, i.e. no CUDA header and no link dependency) and such memory is registered when it is created and unregistered before it is given back. Registration is portable, so it also covers a context created later by `cudaSetDevice`.

Registration follows whichever path supplied the pages: a mapped buffer is registered individually, whereas USM/SVM memory is registered per pool block rather than per `libxs_malloc`, because the pool hands out pointers into a block and registration is page-granular. Blocks that came from plain `malloc` are excluded — those can share a page with unrelated heap data, and that configuration has no OpenCL device to begin with.

`LIBXSTREAM_PIN` chooses a route rather than an amount. The library's own allocations are registered with the vendor runtime under any non-zero setting. A range the caller declared (see below) is staged through a runtime-owned buffer at 1 and registered at 2 and above; the two are alternatives, because they serve different consumers of the memory. Unset is the recommended setting: the route then resolves against the device once one is activated, i.e. staging where the vendor runtime reports no pageable access to host memory and registration otherwise.

There is nothing to enable: a process without CUDA resolves nothing and pays nothing, and a process that links CUDA is one that intends to use it. `LIBXSTREAM_PIN=0` leaves the memory unregistered, which is how the pageable transport is measured. `LIBXSTREAM_PIN=3` goes further and loads the CUDA runtime when the application did not link it. That is off by default on purpose – loading a vendor runtime into a process that never asked for it initializes CUDA, can be slow or fail against a mismatched driver, and may collide with an application managing CUDA itself – but it is the setting a drop-in BLAS replacement needs, since such a process links `libOpenCL` and no CUDA runtime and would otherwise register nothing at all. `LIBXSTREAM_VERBOSE=2` reports how many allocations were offered and how many the CUDA runtime accepted; the partial case is the interesting one, since memory it refused stays pageable and reads as a slow kernel rather than as a slow copy.

Memory the application allocated itself is invisible to the library, so a caller declares such a range with `libxstream_mem_host_pin` (and withdraws it with `libxstream_mem_host_unpin`) to have the route above applied to it; a buffer from `libxstream_mem_host_allocate` must not be declared, since it is page-locked already. Declaring is not device work: the range is recorded without initializing the library or activating a device, and the route is applied to everything declared so far as soon as a device is activated. A process that declares ranges but never touches a device therefore creates no OpenCL context at all. Up to `LIBXSTREAM_MAXNPINS` ranges (32 by default) can be held at a time; a declaration beyond that is rejected rather than silently dropped, and failure is not fatal – the range still transfers, only slower.

Kernel Build

Function	Description
`libxstream_opencl_program`	Compile an OpenCL program from source, file, or binary
`libxstream_opencl_kernel_query`	Extract a named kernel from a compiled program
`libxstream_opencl_kernel`	Convenience: build + extract + release in one call
`libxstream_opencl_kernel_flags`	Assemble combined build flags from params, options, and extras
`libxstream_opencl_defines`	Merge user defines with internal definitions
`libxstream_opencl_flags_atomics`	Generate compiler flags for FP-atomic extensions

Streams, Events, and Timing

Function	Description
`libxstream_opencl_stream`	Find an existing stream for a thread-ID
`libxstream_opencl_stream_default`	Return the device's default (internal) stream
`libxstream_opencl_device_synchronize`	Per-thread device synchronization
`libxstream_opencl_launch`	Launch a kernel (drop-in for `clEnqueueNDRangeKernel`, profiling-aware)
`libxstream_opencl_duration`	Measure elapsed seconds from a `cl_event`

Error Utilities

Function	Description
`libxstream_opencl_strerror`	Map a `cl_int` error code to a string
`libxstream_opencl_error_consume`	Clear and return the last recorded error

Profiling

Two independent facilities, reported at exit on `stderr` and silent unless requested. They are separate because they measure different quantities in different units: a kernel has no byte count and hence no transfer rate, so mixing the rows would invite reading one as the other. Both may be set at once, which is the only case in which both appear.

Variable	Reports
`LIBXSTREAM_PROFILE`	Per-kernel durations (microseconds), one row per distinct kernel
`LIBXSTREAM_PROFILE_MEM`	Transfer rates (GB/s) for H2D, D2H, D2D, and zero-fill

A positive value sets the histogram resolution; a negative value additionally traces every individual sample as it is recorded. Setting either variable enables `CL_QUEUE_PROFILING_ENABLE` on all streams, so profiling is not meant for production runs.

Kernels are identified by `CL_KERNEL_FUNCTION_NAME`, read once per distinct `cl_kernel` handle. Call sites therefore pass no identifier: replacing `clEnqueueNDRangeKernel` with `libxstream_opencl_launch` is sufficient to make a kernel appear in the report.

Samples whose duration spans fewer than `LIBXSTREAM_PROFILE_TICKS` ticks of the device timer (`CL_DEVICE_PROFILING_TIMER_RESOLUT` are counted as discarded rather than recorded, because a rate derived from one or two ticks is quantization noise. The report states the timer resolution and the resulting floor only when samples were actually dropped, and reports `no samples recorded` when a requested profile collected nothing at all — silence there would be indistinguishable from a run that performed no work.

Every value a histogram carries is averaged per sample, never accumulated, so that a bucket's amount and its duration refer to the same single transfer and their ratio is a rate. Accumulating the duration instead divides a per-sample amount by a bucket total, which understates the rate by roughly the number of samples that share the bucket — and equally sized transfers all share one, so the error is largest exactly where the report is most useful.

The headline rate on a transfer row is the histogram's **mode**, not its median. Transfer sizes are commonly multi-modal — a whole operand alongside per-panel blocks, or a zero-fill covering both a small exponent array and a large slice plane — and a median can fall between the clusters and thus describe no observed transfer, whereas the mode always names a bucket that samples landed in. Where a workload issues transfers of one size the two agree, so nothing is lost where the median was already right. Kernel rows report the median duration.

Overlap A row gains one field, `inflight>=`, when its samples overlapped in time:

```
PROF ACC/OpenCL: ID=164984 ZERO=743.5 GB/s inflight>=1.08
PROF ACC/OpenCL: ID=164984 device inflight>=1.04 (3 kernels, 3 transfer kinds)
```

It is the average number of these operations running at once. Intervals cannot sum to more than the time they cover unless some of them ran together, so a ratio above 1 is overlap and nothing else.

The denominator is the **union** of the intervals — the time actually covered — not their extent, which would count the gaps between them. Each histogram folds its own intervals into that union as it goes (`libxs_hist_fold_union`), keeping only the segments a later interval could still merge with. An interval arriving after those have been retired is added whole, since its overlap with retired time can no longer be subtracted; such intervals are counted and the count is reported, and because that can only overstate the union, `inflight` stays a lower bound either way.

The field is absent whenever no overlap is demonstrated, which is the answer in itself: the operations ran one at a time. It is also absent when the overlap is too small to show in the two decimals reported, rather than printing a self-contradictory `1.00`.

A `device` row folds every kernel and every transfer into one union and states how many of each contributed. It reports what no single row can — one kernel overlapping another, or a transfer overlapping a kernel — because a union across

them cannot be assembled from theirs: two rows may cover the same instant, and their totals cannot say whether they do.

A kernel whose every sample was rejected by the accuracy floor still gets a row, saying so, since it was launched and silence would say otherwise.

A negative value additionally traces every interval as `ns=<begin>-<end>` in device-clock nanoseconds, which is enough to reproduce any of this offline.

See Also

- LIBXSTREAM API (`libxstream/libxstream.h`) — public API built on top of this layer
- DBCSR ACC Interface — the DBCSR compatibility shim

Appendix

Presentations

- Stencils as Small Matrix Multiplications
- Practical Ozaki Schemes

Scripts

Helper scripts for building, inspecting, and maintaining LIBXSTREAM.

tool_checkabi.sh

Checks the ABI (Application Binary Interface) stability of the LIBXSTREAM shared library. The script uses `nm` to extract exported symbols from `lib/*.so` (or `lib/*.a` as fallback) and verifies that no previously published symbol has been removed or renamed compared to an earlier baseline (`.abi.txt`). Non-conforming symbol names cause an immediate error.

```
scripts/tool_checkabi.sh
```

NOTE: For full coverage the library must be built with `make STATIC=0 SYM=1` (or `DBG=1`) so that symbol information is present.

tool_checkenvars.sh

Scans the LIBXSTREAM source tree (`src/*.c`) for calls to `getenv`. With `--list` it prints a sorted, deduplicated list of every environment variable used at runtime, separated into LIBXSTREAM-specific (`LIBXSTREAM_*`) and other variables. Without an argument it reports the `LIBXSTREAM_*` variables that no Markdown file documents, which is what the pre-commit hook of the same name runs; `--all` reports the deferred ones as well.

```
scripts/tool_checkenvars.sh --list
scripts/tool_checkenvars.sh
```

tool_opencl.sh

Converts OpenCL kernel files (`.cl`) — and optionally CSV parameter files — into a C header whose string literals can be compiled straight into the host binary. Include guards are stripped by default, and `#include` directives inside kernels are recursively resolved (inline).

```
scripts/tool_opencl.sh [options] infile.cl [infile2.cl ..] [infile.csv ..] outfile.h
```

Flag	Description
`-k`, `--keep`	Keep include guards (normally stripped)
`-b N`, `--banner N`	Copy the first N lines of the first `.cl` file as a banner
`-p DIR`, `--params DIR`	Directory containing CSV parameter files
`-c`, `-d`, `--debug`, `--comments`	Preserve comments in the generated source
`-v`, `--verbose`	Echo the command line

tool_version.sh

Derives the project version from Git tags and revision count. It can emit a plain version string, individual version components, or a complete C header with `#define` guards.

```
## full version string (branch-tag-patch)
scripts/tool_version.sh

## generate a C version header with a given symbol prefix
scripts/tool_version.sh LIBXSTREAM -1

## single component: 1=major, 2=minor, 3=update, 4+=patch
scripts/tool_version.sh LIBXSTREAM 2
```

Argument	Description
`PREFIX`	Symbol prefix used in the generated header (uppercased)
`CMPNT`	`0` (default) — version string; negative — C header; `1`–`3` — major/minor/update; `>3` — patch count
`SHIFT`	Added to the patch count (default: `0`)

Contributing

This is the contribution policy for LIBXS and LIBXSTREAM. Both projects follow the same policy, so the text is project-neutral: it is maintained in LIBXS (`documentation/libxs_contrib.md`) and copied into dependent projects by `make documentation`, the same way `Makefile.inc` is shared. Please edit it in LIBXS — the copy is overwritten whenever the original changes.

The code base is small, long-lived, and read far more often than it is written, so uniformity is what keeps diffs reviewable. When a rule below does not answer your case, the strongest rule is: **match the surrounding code**.

Not every rule is derived from first principles. Some are taste, some are habit, and some are plain superstition. They are the policy regardless: a uniform code base is worth more than an individually optimal choice, and rules that are merely arbitrary are still cheap to follow.

Character Encoding and Whitespace

Files	Encoding
Source, headers, kernels, Makefiles, scripts, YAML, plain text	US-ASCII only
Markdown (`*.md`)	any UTF-8

No Unicode in code or build files — no typographic quotes, no em dashes, no non-breaking spaces. Such characters survive editors, diffs, and generators poorly, and neither the compiler nor `sed` is required to handle them.

Markdown has no such restriction: a spaced em dash (—) instead of --, an en dash for numeric ranges (1–4), and mathematical notation (alpha, <=, ceil(n/2), x, sum) are all welcome wherever they read better than an ASCII transliteration.

Beyond encoding:

- **No tabs**, except where a Makefile requires them.
- LF line endings. CRLF is rejected.

- No trailing whitespace.
- No whitespace before `#` in a preprocessor directive.
- **No French spacing** in either sense: no space before `,`, `;`, `:`, `!`, or `?`, and a single space after a sentence-ending period. This holds for code, comments, commit messages, and documentation alike.
- A script carrying a shebang is executable in the index (mode 755); every other file is 644.

These are enforced mechanically rather than by review. `.pre-commit-config.yaml` combines the standard pre-commit hooks (trailing whitespace, line endings, byte-order marks, shebang and exec-bit consistency, YAML syntax) with the project-specific rules (US-ASCII, tabs, C++ comments, whitespace before `#`, `exit()` in library code, `sed -i` in scripts, `goto`, a declaration in a `for` initializer, banner comments, `--` in a comment, three blank lines, and column 73 in fixed-form Fortran). Install the Git hook once per clone; the same configuration runs in continuous integration, so a violation fails the pull request:

```
scripts/tool_normalize.sh --install   # once per clone
scripts/tool_normalize.sh             # check and fix the whole tree
scripts/tool_normalize.sh src         # or one directory
```

`.editorconfig` keeps most of it from happening in the first place. Data files (`*.csv`, `*.tsv`, `*.dat`) and generated sources are exempt — tabs and line endings are part of their format, and their producer owns them. Of the rules above only the two spacing conventions are checked partially: a space before a comma or semicolon is caught in C sources, the rest is on the author.

The rules that span more than one line are checked by `scripts/tool_checkstruct.py`: a single function exit, blank lines inside and between functions, stacked single-line comments, the shape of a multi-line comment, where an opening brace goes, whether a multi-line `if` or loop is braced, whether the sections of a file come in order, and a constant on the left-hand side. It works on the source text, with comments, literals, and preprocessor directives masked out, so a `return` in `#if` and another in `#else` count as one exit. A parser is not used on purpose: the declarations carry `LIBXS_API` and friends, which a preprocessor-less parser turns into an error node every few lines, and a preprocessed compiler dump no longer says which file a construct came from. `scripts/tool_checkenvars.sh` compares the prefixed variables the source reads against what the documentation mentions.

The policy holds everywhere; the hooks enforce most of it on library code and public headers. A sample or a test that bails out of `main()` early is not the defect a multi-exit library function is, so `single-exit`, `function-gap` and the rest are scoped to `src`, the public headers and the kernels. Four of them are not, because they mislead whoever reads the code wherever it sits: a comment that has come loose from what it documents, a comment whose lines do not carry the `*` that marks them, two closing braces sharing a column while their blocks are nested, and a second blank line inside a function body. Those run over the whole tree, samples and tests included: a contribution is held to the policy wherever it lands, and what that surfaces is listed rather than waived.

Both keep their open findings in a to-do file beside them, `scripts/tool_checkstruct.todo` and `scripts/tool_checkenvars.todo`, rather than in a file-level exclusion. Three properties follow, and each is the point:

- The list is **per rule and per file**, so a file listed for one rule is still checked by the others.
- The list is **per project and not propagated**. The two scripts are policy files that `make documentation` copies from LIBXS; a count lowered in a copy would be reverted, so the state cannot live inside them.
- The list is a **to-do, not a permission**. Each entry carries what it defers, and going the other way fails as well: one finding more than listed is a regression, one less means the entry outlived its findings, and an entry naming a file that no longer exists is reported too. An exclusion that outlives its cause is how a list starts lying.

The list maintains itself in the one direction that is safe. A fix followed by `tool_normalize.sh` lowers the count, and drops the entry when nothing is left; an entry whose file is gone drops too. The run still fails and says what it changed, exactly as the whitespace hooks do, so the smaller list is reviewed and committed rather than applied silently. Upwards never happens: a count that grew, or a new undocumented variable, is a regression and stays an error. `tool_checkstruct.py --counts` prints the table from scratch if a list needs rebuilding wholesale.

C Source File Structure

A translation unit is strictly grouped, in this order:

1. Includes
2. Macros
3. Types (typedefs, struct definitions)

4. Translation-unit variables (file-scope statics and globals)
5. Prototypes, where forward declarations are needed
6. Functions

No interleaving. A macro used only by the last function still belongs in the macro section, and a type or a table used only by the last function belongs in its own. This makes the shape of every file predictable: what it depends on, what it configures, what state it holds, and what it does — in that order. Grouping is also what keeps the sections reviewable: a macro parked next to its first use hides how many of them the file already has.

Three exceptions, and no others:

- A translation unit split into implementation fragments includes those fragments where their code belongs, not at the top. Such an include is a piece of the implementation rather than a dependency, and hoisting it would reorder the definitions it contains. This one is detected rather than judged, by the one property that separates the two: a fragment carries no include guard, because two of them in one translation unit would define everything twice. A dependency is quoted as well where it is a sibling, so the quotes decide nothing.
- A type may sit immediately above the one entry point it parameterizes, when that is what makes the interface readable. This covers an argument or callback type in a public header, not an internal type shared by several functions.
- In a header the last two sections are one. A header is read as a list of declarations, and an inline definition sits in that list: `LIBXS_API_INLINE` interleaved with `LIBXS_API` is the feature group the reader wants, not an interleaving to fix. The reason is the same one that lets a header open a function body on the right. It applies to those two sections only: a macro, a type or a file-scope variable below either of them is still out of order.

A macro that is undefined again is local to the code between the two, the way a variable is local to a block, so it is not a member of the macro section either. `LIBXS_FPRINT_LOAD` in `libxs_math_fprint.h` is defined above the one function that expands it and undefined below, and moving it into the macro section would separate it from its only use and leave the `#undef` reading as noise. The `#undef` has to follow the definition to mean this: the other order is the defensive one, dropping what a platform header may have set, and those macros (`__STRICT_ANSI__`, `_WIN32_WINNT`) are ordinary members of the section.

What sits inside a conditional is a feature test, and a declaration is no exception. A platform header may not declare what it should, so the declaration goes next to the `#define` that calls it and inside the same condition; a type the platform does not provide is defined next to the `#define` that records its availability. `LIBXS_EXTERN void flockfile(FILE*)` in `libxs_sync.h` and the `__int128` typedef in `libxs_macros.h` are both this, and hoisting either one means writing its condition a second time. So inside an `#if`, only a definition is ranked.

That has a prerequisite: a conditional that **wraps the whole file** must not count, or a translation unit that puts `#if defined(__OPENCL)` around everything it contains would have no rules left inside. The test for it is symmetric, since a header ends with a conditional include of its own implementation, which also has nothing after it: nothing that belongs to a section may precede the opener either, and the include guard is the same shape.

The `section-order` check reads the file as a sequence of top-level constructs and reports where their kind steps back to a section the file has already left. It is approximate by design: it ranks what it recognizes and stays quiet about the rest, so it misses cases rather than inventing them. A construct that is nothing but a macro invocation is left unranked, since what it expands to is not in the text, and so are the statements of an included body fragment, whose control flow sits at brace depth zero without being a definition. One backward step is reported once and not once per construct below it, because a single misplaced typedef does not make every macro under it a separate defect.

The shapes ranked out of the ordering are the ones where the text that looks like a section member is not one: the include guard, whose `#define` is the file's own name; anything but a definition inside an `#if`; a prototype immediately above the definition it repeats, which is how a static definition answers `-Wmissing-prototypes`; an include of a guardless implementation fragment; and a macro that is undefined again.

What is left over is judged, one file at a time, rather than guessed at by a rule reading intent – a type above the one entry point it parameterizes has no signal a checker could read. The judgement is recorded where it stays visible: in the to-do list while the file has structure the rule must keep watching, and in the script's permanent list only where the accepted shape is the one thing that rule could report in that file. `libxs_macros.h` is the second kind, since its standard includes must follow the feature-test macros that configure them, and macros and those includes are all it holds.

Every file carries the SPDX license header (BSD-3-Clause) verbatim as found in existing files.

C Dialect

C89 (ANSI C) in general. The pre-submit configuration (`PEDANTIC=2`) compiles with `-std=c89`, so a C99-only construct breaks the build for everyone else even when it compiles for you:

- Declarations at the beginning of a block, before any statement. No declarations mixed into the middle of a block, and no declaration in a `for` initializer.
- `/* ... */` comments only.
- No variable-length arrays, compound literals, designated initializers, `restrict`, or `//`-style line continuation tricks.
- Newer facilities are reached through the macros and typedefs the public headers already provide, not by raising the dialect locally.

Where a C99 (or later) construct is genuinely required, it is guarded and confined, in the same style as the existing guards.

OpenCL kernels (`*.cl`) follow every rule above except the dialect. They are compiled as OpenCL C, which is C99, so a declaration in a `for` initializer is correct there and the hooks exempt kernels from that one rule. Everything else holds unchanged: US-ASCII, no tabs, `/* ... */` comments only, a single function exit, two blank lines between functions, capitalized macros, and the SPDX header. A kernel is source, not data.

A 64-bit integer is `uint64_t` or `int64_t`. They are exactly 64 bits wide rather than merely at least that wide, they add no name of ours to the API, and they cost the user nothing: `libxs_macros.h` already includes `<stdint.h>` and `<inttypes.h>`, so every consumer of the public headers has them, and the API already returns `uint64_t` from `libxs_hilbert` and `libxs_morton`. `long long` is kept only where something outside the project spells it — the Intel intrinsics take `long long*` (`_mm256_i64gather_epi64`), the `__atomic_*_8` builtins take `long long`, and a value printed with `%llu` is cast to `unsigned long long`, which is portable without composing the format string out of `PRIu64`. Those are the reasons `-Wno-long-long` is set for a pedantic build, and they are the only ones: new code does not reach for `long long` to hold a number of its own.

Functions

- **A function has a single exit.** No early `return`, no multiple return paths, no `goto`.
- **No trailing underscore on a parameter or a local**, in a definition or a declaration. That mark is reserved for a variable a *macro* declares, so that such a name cannot collide with one the caller already has in scope; a function wearing it anywhere takes the mark away from the one thing it is for. Passing an underscored name *to* a function is a different matter and fine: inside a macro body that is exactly what the macro's own local is for.
- Use a `result` variable, gate subsequent work on `EXIT_SUCCESS == result`, and return `result` at the single exit point.
- Constants go on the left-hand side of a comparison (`EXIT_SUCCESS == result`, `NULL != ptr`), which turns an accidental assignment into a compile error.
- Two blank lines between function definitions.
- At most one blank line inside a function body.

```
int example(const void* input, void** output)
{
  int result = EXIT_SUCCESS;
  if (NULL == input || NULL == output) result = EXIT_FAILURE;
  if (EXIT_SUCCESS == result) {
    result = prepare(input);
  }
  if (EXIT_SUCCESS == result) {
    result = finish(input, output);
  }
  return result;
}
```

Macros

A macro is capitalized, and so are its parameters: `LIBXS_ALIGN(POINTER, ALIGNMENT)`, not `LIBXS_ALIGN(pointer, alignment)`. The parameters are the part that is easy to forget, and they are the part that matters at the point of use: a capitalized argument is what tells the reader that the expression may be evaluated more than once. **A parameter carries no**

trailing underscore: that marks a variable the macro declares itself, and the two must stay apart, since the whole point of the underscore is to say "this name is mine, not yours".

A variable a macro declares is the other way round: lowercase, with a trailing underscore, and ideally prefixed by the macro's own name.

```
#define MACRO() { int macro_i_ = 0; }
```

The capitalization tells the reader which names come from the call site and which the macro invented; the trailing underscore and the prefix are what keep the invented one from colliding with a variable the caller already has in scope. A macro that declares plain `i`, `s`, or `p` is a trap for whoever expands it next to their own `i`. The members of an aggregate the macro declares are not locals — in `union { float v; float a[8]; } u_` it is `u_` that carries the underscore — and a `*_DECL(A)` macro that declares a variable named by its own parameter keeps the caller's spelling.

Two kinds of macro name are lowercase on purpose, and both are exempt:

- A trailing lowercase segment is a token pasted onto the name — a type (`LIBXS_TYPECHAR_double`), a lock kind (`LIBXS_LOCK_ACQUIRE_spin`), an address space. It has to match the spelling of what it names.
- A macro standing in for something that is not ours keeps that spelling: a compiler builtin (`__builtin_nan`), a language keyword the OpenCL-on-CPU path defines away (`kernel`, `restrict`), a foreign API (`offloadSuccess`), or the lowercase alias a paste target needs (`libxs_crc32_b8`).

Comments

- **A comment adds value on top of the code.** Never document what the code obviously does — the code already says that. Document what it cannot say: why this way, what breaks otherwise, which assumption is being relied on.
- Prefer no comment at all. Then prefer one line.
- **Never stack comments.** Two comments with no code between them are either one comment — make it one line, or a block if it earns the size — or they describe different subjects, and then the code each one describes belongs between them. **A blank line between them does not separate them**: it is still two comments and no code. This covers blocks as much as single lines; the license header, being the file's leading comment, is exempt.
- **Single-line comments by default.** A multi-line comment has to earn its size: it is reserved for what is absolutely necessary, i.e., a risk or a trap worth spelling out — typically something that has already been gotten wrong once and would be gotten wrong again without the warning. Everything else is one line or nothing.
- API documentation in public headers is the other place a multi-line block belongs.
- **A multi-line block carries a `*` on every line**, under the one in the opening `/*`, the closing line included. Two shapes satisfy that and both are accepted outside a header: the block that opens on its own line, and the compact one whose text starts on the opening line and ends on the closing one. What it rules out is a third shape, a hanging indent with no stars, where nothing marks the lines as comment and a reader scanning the left edge loses them.
- **In a header, a block at file scope opens and closes on its own line.** The reason is the API rather than the comment, and it is the one that lets a header open a function body on the right: the declarations are read as a list, and a block starting and ending mid-line breaks the rhythm of that list. Inside a function body, and in an implementation unit, either shape is fine. Measured before requiring it: 452 blocks at header file scope already open on their own line and 15 do not.
- `/* ... */` only. **C++ comments (`//`) are rejected** in `.c` and `.h`.
- **No decorative or banner-style comment blocks.** No `/*==== Section ====*/`, no boxes, no ASCII rules. The license header is the sole exception.
- Never write `--` inside a comment (the ASCII rule keeps the em dash out, and a double hyphen reads as a typo); rephrase instead.
- These rules govern the comments a change **writes**. Do not restyle existing comments along the way: one that already earned its size keeps it, and reflowing a file's comments buries the actual change exactly as a bulk reformat does.

```
/* one line is the default, and most comments need no more */

/**
 * A block earns its size by naming a trap: what was already gotten wrong once,
 * and would be gotten wrong again without the warning.
 */
```

Blank Lines

- Never three or more consecutive blank lines, anywhere.
- Exactly two blank lines separate function definitions in a `.c` or `.cl` file. **A header may use one**, and the files that carry small `LIBXS_API_INLINE` definitions are uniform about it: `libxs_gemm.h` and `libxs_token.h` use one throughout, `libxs_math.h` for sixteen of its twenty-one. One or two, but not a mixture within a file, which is the surrounding code a change there has to match.
- At most one blank line separates logical blocks inside a function. Two is what separates the functions themselves, so two never appear inside one.

Formatting

A `.clang-format` file is present (LLVM-based, 2-space indent, 96-column limit, never tabs), and `scripts/tool_clangformat.sh` selects the newest available version of the tool.

Do not bulk-reformat as part of a change. Recent clang-format versions reflow entire files, which buries a small change in hundreds of unrelated lines and makes review impossible. This is why clang-format is deliberately absent from the hook set. Format new and edited code by hand to match the file around it; the formatter is a maintenance tool, run deliberately and committed on its own.

Do not mix reformatting, renaming, and behavioural change in one commit.

Where the opening brace goes. It stays on the line of the construct it belongs to — `if (0 < n) {`, `for (...) {`, `} else {` — with two exceptions, and both are checked:

- **A function body opens on its own line in an implementation unit**, a `.c` or a `.cl` file: 1971 definitions do it and 78 do not.
- **A brace whose parentheses were broken across lines opens on its own line**, because the brace is then what tells the reader the condition has ended:

```
if (0 == first &&
    0 != second)
{
  ...
}
```

In a header the brace may trail the signature, and it usually should. What rules a header is a readable API: the declarations are read as a list, and an inline definition sits in that list. It is also outside the ABI and meant to stay a small tool, so a body long enough to want the brace on a line of its own needs a reason — `libxs_gemm.h` has one. Both forms are accepted there, which is the difference from an implementation unit; a broken parameter list still takes the brace onto its own line.

A bare block has no construct line to sit on, so nothing is required of it: `{ int scope_ = n;` and a brace alone on the line are both in use and both fine. A struct, a union and an initializer are left alone as well.

Whatever an `if`, an `else` or a loop controls stays on the keyword's line, or it is braced. Once the construct reaches a second line the braces are what say where it ends, and that includes the case where only the condition wrapped:

```
while (npos < (int)ctx->text_size
  && 0 != isspace(ctx->text[npos]))
{
  ++npos;
}
```

Each keyword is judged on its own, so an `if` with a braced body and a one-line `else` is two decisions rather than one. An `else if` is the inner `if` and is judged there.

Two more places keep the brace on its own line, and the check knows both. One is a construct whose line is followed by a preprocessor directive: the line above the brace is then `#endif`, and moving the brace up would carry it into the branch. The other is a header that is included inside a function body, where the file's own control flow sits at brace depth zero and is not a definition.

```
#if defined(LIBXS_CPUID_ARM_MODEL_FALLBACK)
  if (NULL != info)
#endif
  {
    ...
  }
```

A directive in that position also excuses the braces themselves: the keyword and what it controls then belong to different configurations, and one pair of braces cannot serve both.

Indentation is not reformatted by a hook, but one thing about it is checked: **two closing braces in a row must step left.** Sharing a column means they close blocks that are nested, so a level is missing from the indentation even though the braces balance and the compiler is content. The check is local and makes no assumption about the indent unit, and a preprocessor directive between the two resets it, because which brace belongs to which block then depends on the configuration. `} else {` is not a closer for this purpose: it reopens, so the next closer legitimately shares its column.

Library Code

- Library code does not terminate the process: no direct `exit(...)` in `src/`. Return a status and let the caller decide. The macro that wraps the one unavoidable case is the single exception.
- Environment variables carry the project's own prefix (`LIBXS_*` or `LIBXSTREAM_*`). `scripts/tool_checkenvars.sh --list` lists what the source reads, ours and foreign.
- **Header-only mode must keep working.** The amalgamated header (`*_source.h`, or the corresponding `-D*_SOURCE`) has to be includable from multiple translation units, so a new file-scope symbol in `src/*.c` needs internal linkage or the established macro treatment.
- Fix the cause, not the symptom. When a sample, test, or dependent project runs into a limitation of the library, change the library rather than working around it at the call site.

Fortran

Fortran sources are **fixed-form** (`.f`). Free-form (`.f90`) is not used; please do not propose converting them.

- Statements occupy columns 7 to 72. Columns 73 and beyond are ignored, so text reaching column 73 is **silently truncated** rather than diagnosed.
- A continued line carries `&` in column 6. Sources additionally place a trailing `&` in column 73, so the same file also reads as free-form; that marker is the only thing allowed at column 73.
- `.fi` files are included into fixed-form sources and follow the same rules.

Scripts

- POSIX-portable shell. `sed -i` is rejected: it is not portable (macOS).
- Shell scripts pass `shellcheck`, which the hooks run.
- Python is formatted with `black -l79` and passes `flake8`; `mypy` covers the tooling under `scripts/` and `.theme/`, not sample code.
- Where a hook carries an exclusion, it names the open findings it defers. An exclusion is a backlog item, not a permission.

Documentation

Documentation is terse and written for the person *using* the code, not for the person who wrote it.

- **Every `README.md` is user documentation**: what the thing does, how to enable it, how to run it, which environment variables and build knobs exist. Nothing more.
- Insights stay out. Design rationale, derivations, and performance analysis do not belong in a README — the single exception being a surprising usability implication the user has to know about (a knob that silently changes accuracy, a mode that only works on one vendor). If it does not change how someone uses the code, it is not user documentation.

- A short *why* belongs in the commit message.
- Document every new environment variable. A variable that `tool_checkenvars.sh` reports but the documentation does not mention is a defect, and the hook of the same name says so.
- A new page under `documentation/` needs a `nav` entry in `mkdocs.yml`.

Markdown may use any UTF-8, but the PDF is produced through LaTeX, which cannot render an arbitrary glyph. `PDF_UTF8_SED` in `Makefile.inc` transliterates a known set (Greek letters, arrows, comparison and set operators, ceiling and floor brackets). If `make documentation` fails on a character, add it there rather than removing it from the text.

Parts of `documentation/` are **generated**, and editing the output is lost work: the landing page comes from `README.md`, the development page from `scripts/README.md`, one page per sample from `samples/*/README.md`, one page per test from `tests/*.c`, this page from LIBXS, and the PDFs from all of the above. Version and amalgamated headers are generated too. Edit the source, then:

```
make documentation   # PDFs
make mkdocs          # serve the site with live reload
make mkslides        # serve a slide deck (SLIDES=<topic>)
```

The Makefile is authoritative about what is generated from what.

Build Systems

GNU Make is primary. The `Makefile` and the shared `Makefile.inc` define the defaults, the knobs, and the behaviour; they are the reference. **CMake is secondary**: it has to produce the same artifacts, but it does not get to define anything.

- A new source file, build knob, or changed default lands in the Makefile first, then in `CMakeLists.txt`. `CMakeLists.txt` mirrors the Makefile's defaults and says so where it matters — when a default moves, move both.
- Every step is taken to keep the two interchangeable *from the consumer's side*. In particular, `make` writes the files a CMake consumer expects — the package configuration under `lib/cmake/<project>/` alongside the pkg-config `.pc` files — so a tree built and installed with GNU Make is usable via `find_package()` without CMake ever having run. GNU Make pretending to be CMake is a feature, not a workaround.
- Continuous integration builds and installs both ways and then consumes the result via `find_package()` and pkg-config. A change that only works in one of the two is incomplete.

Building and Testing

Build without `DBG=1` by default. Debug builds are `-O0`, which makes any runtime-bearing sample prohibitively slow — never measure performance with one.

```
make -j $(nproc)                    # default build
make -j $(nproc) test               # test suite
make -j $(nproc) DBG=1 PEDANTIC=2   # correctness check before submitting
```

`PEDANTIC=2` enables strict warnings and `ANALYZE=1` runs the compiler's static analyzer; where a project ships `scripts/tool_analyze.sh`, that runs cppcheck on top. A change is expected to be warning-free under `DBG=1 PEDANTIC=2`, because that is what continuous integration builds: GCC, Intel oneAPI, and macOS, covering release and strict debug configurations as well as a header-only build compiled as C++. See `.github/workflows/` for the exact matrix.

`SANITIZE=<kind>` passes `-fsanitize=<kind>` to both the compiler and the linker. ThreadSanitizer needs no options file: `__tsan_default_options` and `__tsan_default_suppressions` in `libxs_main.c` carry what a run of the library needs, and the environment is parsed after them, so `TSAN_OPTIONS` still overrides either one. They are inert in a shared-library build, where the runtime's own weak definition is found first, and that build needs the environment instead.

```
make SANITIZE=thread
setarch -R ./sample.x   # ASLR off, or the runtime aborts before main()
```

Some kernels abort with `unexpected memory mapping` unless ASLR is off, hence the `setarch`. Exit code 66 means races were reported, not that the program failed.

GNU libgomp is not instrumented, so a race whose two accesses sit on opposite sides of an `omp parallel` region is reported even though the region's own barrier separates them; a fifteen-line correct program reproduces it. The compiled-in `ignore_noninstrumented_modules=1` cuts the volume about fivefold but not the class. **Triage by region**

position, not by thread: a report is that artifact unless *both* stacks carry an `_omp_fn` frame. Thread identity is not the test, since the main thread is a member of the team and takes part in the rendezvous. Suppressing the class is not possible, because those stacks run through the code under test; building with Clang against a libomp that has TSan support (it ships `libarcher.so`) is what makes the region boundaries visible.

ABI and Versioning

The version derives from Git tags and `version.txt`; the shared library carries an SOVERSION. `scripts/tool_checkabi.sh` compares exported symbols against the recorded baseline. Do not remove or rename a published symbol — add a new one instead. Run the checker on a build that has symbol information:

```
make STATIC=0 SYM=1
scripts/tool_checkabi.sh
```

A symbol name outside the project's namespace is an error, not a warning.

Commits and Pull Requests

- One concern per commit. Keep unrelated formatting out of it.
- Subject line: short, capitalized, no trailing period, e.g. `Improved device memory allocation`. An area prefix is fine: `CMake: updated to test kernels`.
- Explain *why* in the body when the subject cannot carry it.
- Contributions are accepted under the project's BSD-3-Clause license.

Before submitting:

```
scripts/tool_normalize.sh --install       # once per clone, then automatic
scripts/tool_normalize.sh                 # whitespace, encoding, lint
make -j $(nproc) DBG=1 PEDANTIC=2 test    # strict build and tests
scripts/tool_checkabi.sh                  # only if public symbols changed
```

Workflow with Assistants

If an AI assistant is used, the same policies apply, plus two more:

- Discuss design options and trade-offs before implementing.
- Present options as inline text, not as interactive choice widgets.